



**WYŻSZA SZKOŁA
INFORMATYKI i ZARZĄDZANIA**
z siedzibą w Rzeszowie

KOLEGIUM INFORMATYKI STOSOWANEJ

Kierunek: INFORMATYKA

Grupa: 3IIZ/2026 L2

Wiktor Nicpoń

Nr albumu studenta: 71343

Program Sklep

Pod kierunkiem: mgr inż. Ewa Żesławska

PROJEKT

Rzeszów 2026

Spis treści

Wstęp	2
1 Wymagania projektu	3
1.1 Wymagania funkcjonalne	3
1.1.1 Zarządzanie produktami	3
1.1.2 Zarządzanie zamówieniami	3
1.1.3 Autoryzacja i uwierzytelnianie	3
1.2 Wymagania niefunkcjonalne	3
2 Opis struktury projektu	5
2.1 Struktura plików i klas	5
3 Opis techniczny projektu	7
3.1 Środowisko i konfiguracja	7
3.1.1 Wymagania systemowe i programowe	7
3.1.2 Wymagania sprzętowe	7
3.2 Mechanizm przechowywania danych	8
4 Harmonogram prac	9
4.1 Diagram Gantta	9
5 Repozytorium i system kontroli wersji	10
6 Prezentacja warstwy użytkowej	11
6.1 Sprzedawca - Zarządzanie systemem	11
6.1.1 Zarządzanie magazynem	12
6.1.2 Wyświetlenie historii zamówień	13
6.1.3 Zarządzanie klientami	14
6.2 Klient - Scenariusze zakupowe	15
6.2.1 Scenariusz Klienta Detalicznego	15
6.2.2 Scenariusz Klienta Hurtowego	18
7 Podsumowanie	22
7.1 Wnioski z realizacji projektu	22
7.2 Plany rozbudowy	22
Bibliografia	23

Wstęp

Celem niniejszego projektu było zaprojektowanie i zaimplementowanie aplikacji konsolowej w języku C#, służącej do kompleksowej obsługi sklepu odzieżowego. System wspiera dwa kluczowe obszary działalności: perspektywę klienta (zakupy, koszyk) oraz perspektywę sprzedawcy (zarządzanie magazynem, administracja użytkownikami).

Aplikacja kładzie nacisk na ergonomię użytkowania w środowisku tekstowym oraz trwałość danych, realizowaną poprzez autorski system zapisu do plików. Kluczowym wyróżnikiem jest implementacja logiki biznesowej rozróżniającej klientów detalicznych i hurtowych. W procesie implementacji ściśle wykorzystano paradygmaty programowania obiektowego, co pozwoliło na stworzenie modularnej i łatwej w rozbudowie struktury kodu. Dodatkowo system wyposażono w mechanizmy walidacji danych wejściowych, zapewniające integralność informacji oraz stabilność działania aplikacji w przypadku błędów użytkownika.

Rozdział 1

Wymagania projektu

1.1 Wymagania funkcjonalne

Wymagania funkcjonalne określają zakres operacji, jakie system musi umożliwiać użytkownikom końcowym. Na ich podstawie zaprojektowano strukturę aplikacji oraz sposób jej działania. Obejmują one zarówno funkcje dostępne dla klientów, jak i operacje administracyjne realizowane przez sprzedawcę. Wymagania te stanowią podstawę do poprawnej realizacji procesów sprzedażowych w systemie.

1.1.1 Zarządzanie produktami

System umożliwia sprzedawcy zarządzanie asortymentem sklepu poprzez dodawanie nowych produktów oraz modyfikację istniejących pozycji. Każdy produkt posiada określone cechy, takie jak nazwa, cena oraz ilość dostępna w magazynie. Wprowadzone zmiany są zapisywane w systemie przechowywania danych, co zapewnia aktualność informacji. Funkcjonalność ta pozwala na bieżącą kontrolę oferty sklepu.

1.1.2 Zarządzanie zamówieniami

Proces zakupowy umożliwia klientowi przeglądanie katalogu produktów oraz dodawanie wybranych pozycji do koszyka. Zawartość koszyka może być modyfikowana przed finalizacją zamówienia. Po zatwierdzeniu koszyka system zapisuje dane zamówienia oraz aktualizuje stany magazynowe. Każde zamówienie jest przypisane do konkretnego użytkownika.

1.1.3 Autoryzacja i uwierzytelnianie

System obsługuje mechanizmy rejestracji oraz logowania użytkowników. Wyróżniono trzy role użytkowników, z których każda posiada inny zakres uprawnień. Sprzedawca ma dostęp do funkcji administracyjnych, natomiast klienci mogą realizować proces zakupowy. Klient hurtowy otrzymuje automatycznie naliczany rabat w wysokości 10%.

1.2 Wymagania niefunkcjonalne

Wymagania niefunkcjonalne opisują cechy systemu wpływające na jego jakość i komfort użytkowania. Obejmują one aspekty takie jak wydajność, trwałość danych oraz użyteczność interfejsu.

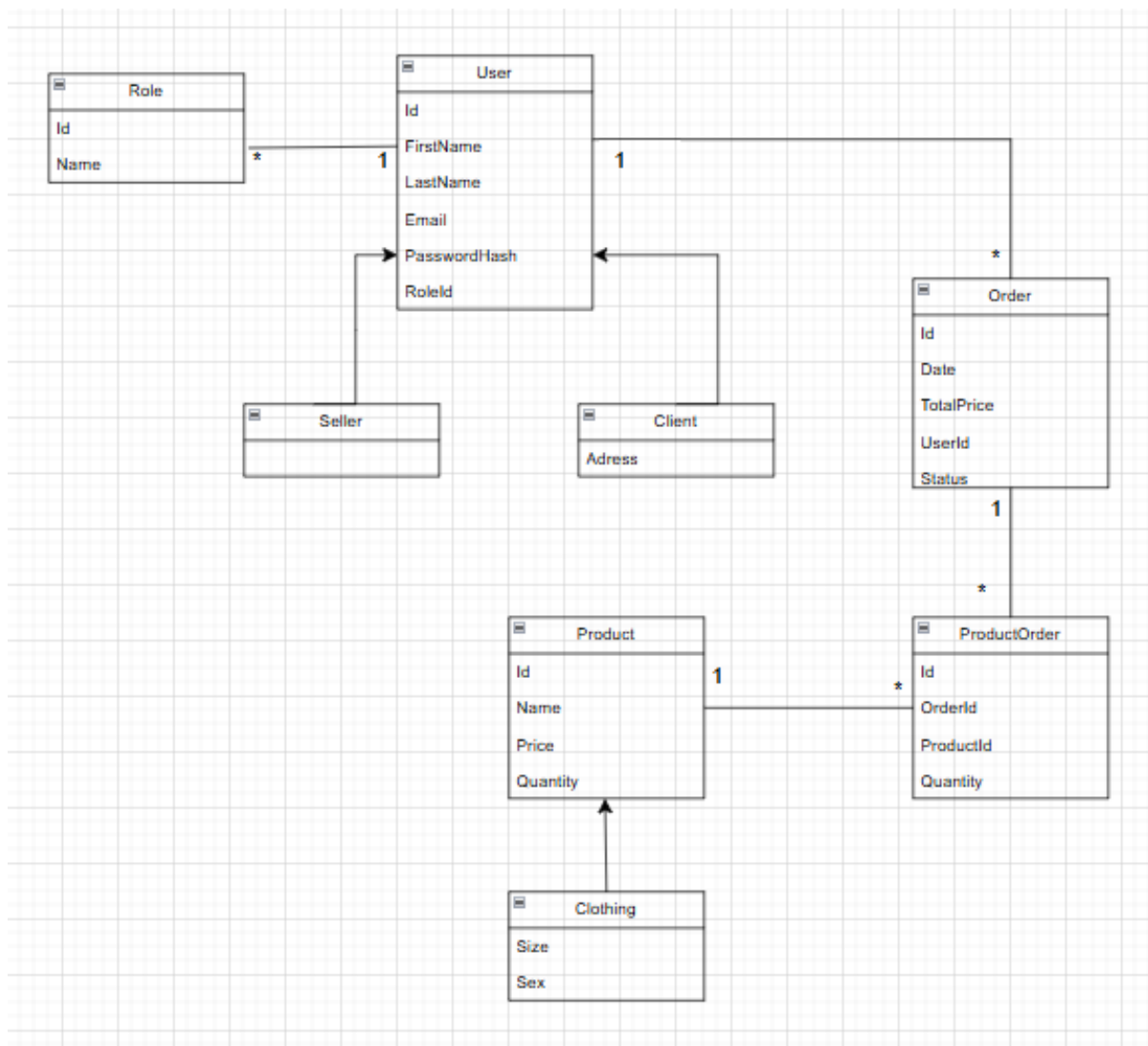
- **Wydajność:** System powinien reagować szybko na działania użytkownika. Zastosowanie plików tekstowych jako mechanizmu przechowywania danych pozwala na sprawne przetwarzanie informacji bez dużego obciążenia systemu.
- **Trwałość danych:** Dane aplikacji są zapisywane w plikach tekstowych w katalogu Data, co umożliwia ich zachowanie pomiędzy kolejnymi uruchomieniami programu. Każda istotna operacja skutkuje aktualizacją odpowiednich plików.
- **Użyteczność:** Interfejs konsolowy został zaprojektowany z myślą o czytelności i prostocie obsługi. Wykorzystano kolory oraz komunikaty tekstowe w celu ułatwienia nawigacji i interpretacji informacji.

Rozdział 2

Opis struktury projektu

2.1 Struktura plików i klas

Struktura projektu została zaprojektowana w sposób modułowy, co umożliwia czytelne rozdzielanie odpowiedzialności poszczególnych elementów aplikacji. Architektura systemu opiera się na logicznym podziale na warstwy, co ułatwia organizację kodu oraz jego późniejszą rozbudowę. Na rysunku 2.1 przedstawiono diagram klas ilustrujący strukturę projektu oraz zależności pomiędzy kluczowymi elementami systemu.



Rysunek 2.1: Diagram klas projektu

Główne elementy struktury projektu obejmują:

- **Models:** Zawiera klasy reprezentujące podstawowe obiekty systemu, takie jak użytkownik, produkt oraz zamówienie, które odzwierciedlają dane przetwarzane przez aplikację.
- **Repositories:** Odpowiada za obsługę zapisu i odczytu danych z plików tekstowych, pełniących rolę uproszczonej bazy danych.
- **Helpers:** Zawiera funkcje pomocnicze wspierające działanie aplikacji, w tym obsługę koszyka oraz walidację danych wprowadzanych przez użytkownika.

Rozdział 3

Opis techniczny projektu

3.1 Środowisko i konfiguracja

Aby zapewnić prawidłowe działanie oraz możliwość kompilacji aplikacji "System Zarządzania Sklepem", stacja robocza musi spełniać określone wymagania programowe i sprzętowe. Poniżej przedstawiono szczegółową specyfikację środowiska, w którym projekt został stworzony i przetestowany.

3.1.1 Wymagania systemowe i programowe

Projekt został zrealizowany w oparciu o nowoczesny ekosystem firmy Microsoft, co gwarantuje stabilność i bezpieczeństwo.

- **System operacyjny:** Zalecanym środowiskiem jest **Microsoft Windows 11**, jednak aplikacja zachowuje pełną kompatybilność również z systemem Windows 10 (wersja 2004 lub nowsza).
- **Środowisko programistyczne (IDE):** Do wytworzenia kodu wykorzystano **Microsoft Visual Studio 2022** (wersja Community lub Professional).
- **Framework:** Aplikacja bazuje na platformie **.NET 10.0 LTS** (Long Term Support), co zapewnia długoterminowe wsparcie techniczne oraz dostęp do najnowszych funkcjonalności języka C#.
- **Szablon projektu:** Wybrano szablon "**Aplikacja konsoli**" (**Console Application**), który jest optymalny dla systemów operujących na interfejsie tekstowym, zapewniając minimalny narzut wydajnościowy.

Opisane środowisko programistyczne zapewnia możliwość bezproblemowej kompilacji, uruchamiania oraz dalszego rozwoju aplikacji zarówno w warunkach laboratoryjnych, jak i domowych.

3.1.2 Wymagania sprzętowe

Ze względu na konsolowy charakter aplikacji oraz zastosowanie lekkiej bazy danych opartej na plikach tekstowych, wymagania sprzętowe są minimalne:

- **Procesor:** Jednostka o architekturze x64 lub ARM64 o taktowaniu min. 1 GHz.

- **Pamięć RAM:** Minimum 512 MB dostępnej pamięci operacyjnej (zalecane 4 GB dla komfortowej pracy z Visual Studio).
- **Przestrzeń dyskowa:** Aplikacja wymaga zaledwie ok. **50 MB** wolnej przestrzeni dyskowej. Wartość ta obejmuje skompilowane pliki wykonywalne (.exe, .dll) oraz katalog Data przechowujący pliki tekstowe bazy danych.

3.2 Mechanizm przechowywania danych

Zastosowano autorski mechanizm przechowywania danych oparty na plikach tekstowych (.txt) z separatorem średnikowym. Rozwiązanie to pełni rolę uproszczonej bazy danych i umożliwia trwały zapis informacji pomiędzy kolejnymi uruchomieniami aplikacji.

- **users.txt** – Plik przechowujący dane użytkowników oraz informacje o ich rolach w systemie.
- **products.txt** – Plik zawierający katalog produktów dostępnych w sklepie.
- **orders.txt** – Plik przechowujący historię zrealizowanych transakcji.

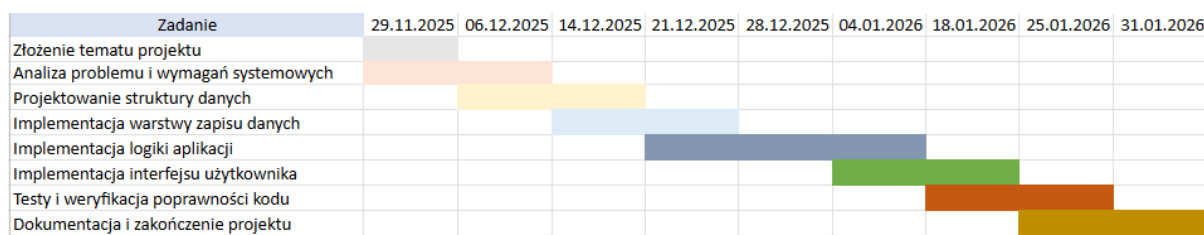
Rozdział 4

Harmonogram prac

4.1 Diagram Gantta

Poniższy wykres prezentuje rozkład prac związanych z realizacją projektu w czasie. Harmonogram obejmuje wszystkie kluczowe etapy tworzenia aplikacji, począwszy od analizy wymagań, poprzez projektowanie oraz implementację systemu, aż po testowanie i przygotowanie dokumentacji końcowej.

Na rysunku 4.1 przedstawiono diagram Gantta ilustrujący kolejność realizacji poszczególnych zadań oraz czas ich trwania. Część etapów była realizowana równolegle, co pozwoliło na efektywne wykorzystanie dostępnego czasu. Takie podejście odzwierciedla rzeczywisty przebieg prac projektowych oraz umożliwiło terminowe zakończenie projektu.



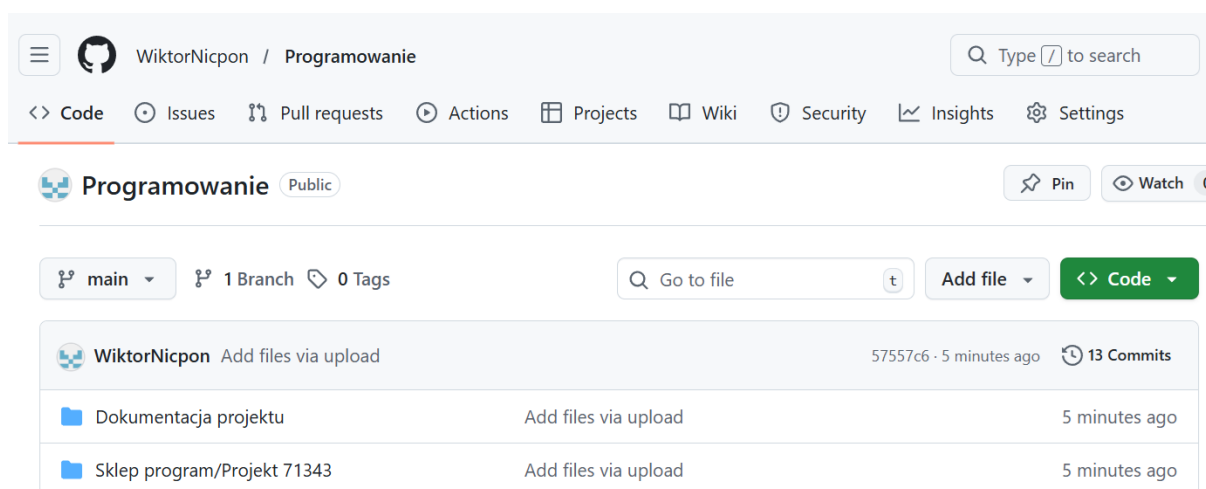
Rysunek 4.1: Diagram Gantta

Rozdział 5

Repozytorium i system kontroli wersji

W trakcie realizacji projektu wykorzystano system kontroli wersji Git, który umożliwiał bieżące zarządzanie kodem źródłowym oraz śledzenie historii wprowadzanych zmian. Zastosowanie repozytorium pozwalało na systematyczne zapisywanie kolejnych etapów prac, a także ułatwiało testowanie aplikacji oraz eliminację błędów pojawiających się w trakcie implementacji. Kod źródłowy był wersjonowany lokalnie, a następnie publikowany w zdalnym repozytorium.

Repozytorium projektu zostało udostępnione w serwisie GitHub, co umożliwia wgląd w strukturę projektu oraz historię commitów. Podstawowa struktura repozytorium została przedstawiona na rysunku 5.1. Repozytorium dostępne jest pod adresem: <https://github.com/WiktorNicpon/Programowanie.git>



Rysunek 5.1: Struktura repozytorium projektu w serwisie GitHub

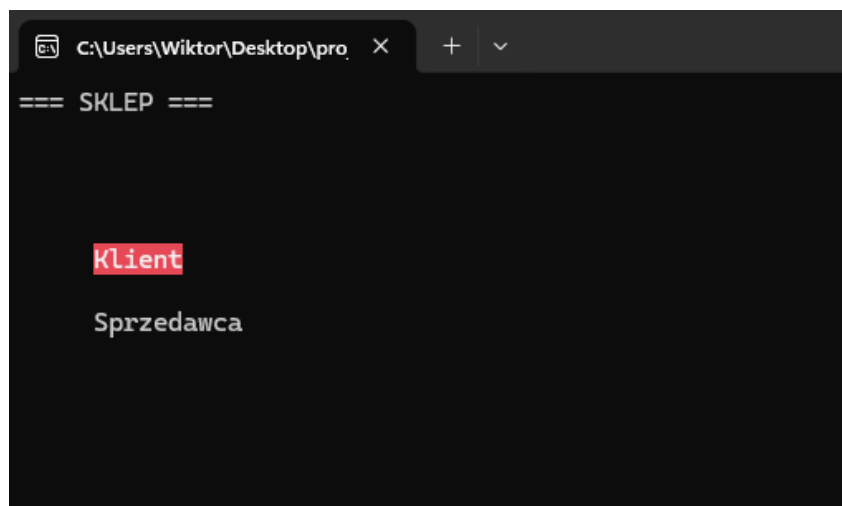
Rozdział 6

Prezentacja warstwy użytkowej

Aplikacja sklepu została zaprojektowana jako rozwiązanie działające w trybie konsolowym. Całość funkcjonalności, obejmująca zarówno mechanizmy przetwarzania danych, jak i sposób prezentacji informacji użytkownikowi, została opracowana w języku C#. Obsługa programu odbywa się w środowisku tekstowym, bez zastosowania interfejsu graficznego, a komunikacja z systemem realizowana jest wyłącznie za pomocą klawiatury.

W kolejnej części rozdziału zaprezentowano wybrane zrzuty ekranu ilustrujące kluczowe elementy działania aplikacji. Materiały te mają na celu przybliżenie sposobu interakcji użytkownika z systemem oraz przedstawienie organizacji interfejsu konsolowego.

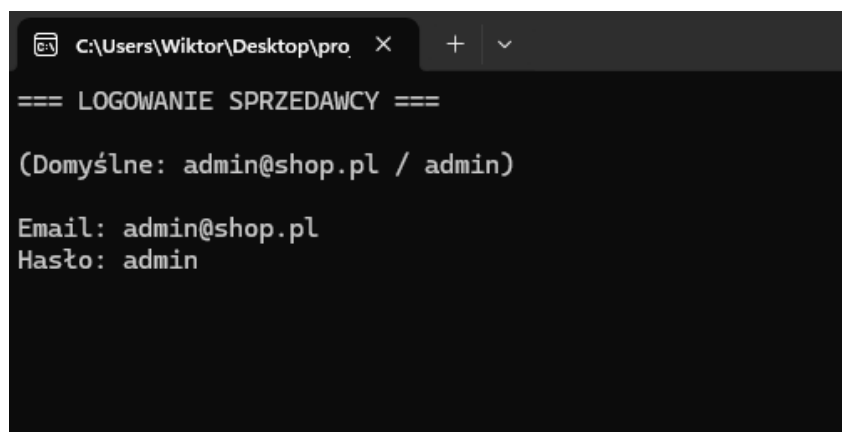
Po uruchomieniu programu wyświetlany jest ekran startowy, który umożliwia wybór trybu pracy aplikacji. Użytkownik może zdecydować, czy chce korzystać z systemu w roli sprzedawcy, czy jako klient, co determinuje dostępny zakres funkcjonalności.



Rysunek 6.1: Ekran główny aplikacji

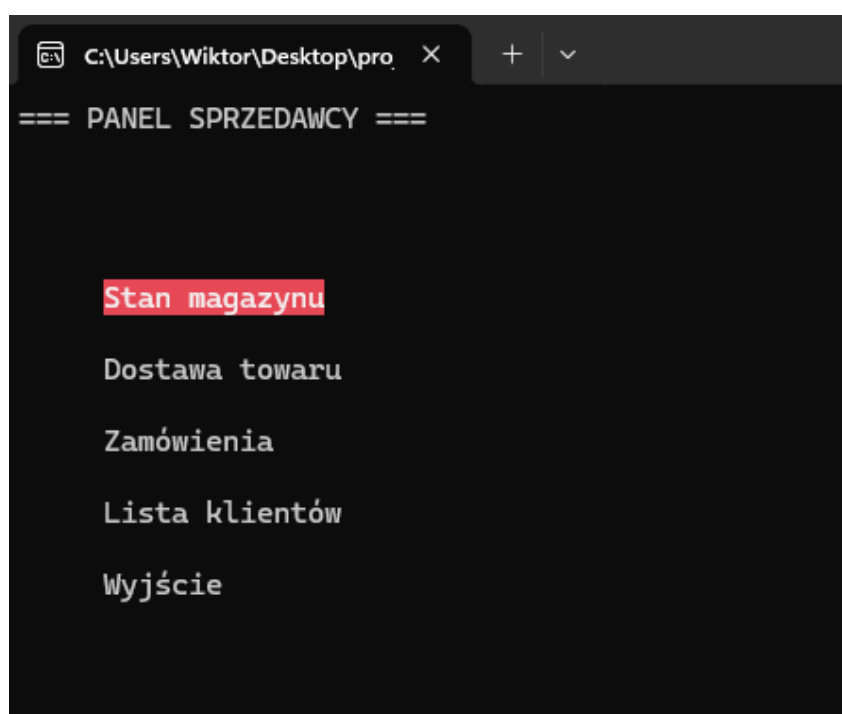
6.1 Sprzedawca - Zarządzanie systemem

Logowanie do panelu sprzedawcy wymaga uprawnień administratora (Rysunek 6.2).



Rysunek 6.2: Logowanie do panelu sprzedawcy

Sprzedawca po zalogowaniu ma dostęp do kluczowych funkcji zarządczych (Rysunek 6.3).



Rysunek 6.3: Menu główne sprzedawcy

Menu sprzedawcy umożliwia dostęp do wszystkich funkcji administracyjnych systemu. Z jego poziomu możliwe jest zarządzanie magazynem, przeglądanie historii zamówień oraz administracja kontami użytkowników.

6.1.1 Zarządzanie magazynem

Sprzedawca może weryfikować stany magazynowe. Poniżej przedstawiono proces dodawania nowego towaru. Najpierw widoczny jest stan początkowy (Rysunek 6.4), następnie dodanie produktu "Kurtka" (Rysunek 6.5), a na końcu zaktualizowana lista produktów (Rysunek 6.6).

```
C:\Users\Wikt\or\Desktop\pro X + v
=== STAN MAGAZYNU ===
ID: 1 | T-Shirt (L) | Ilość: 1 | Cena: 50
ID: 2 | Jeans (32) | Ilość: 5 | Cena: 120
```

Rysunek 6.4: Stan magazynu przed dostawą

```
C:\Users\Wikt\or\Desktop\pro X + v
=== PRZYJĘCIE TOWARU ===
ID: 1 | T-Shirt (L) | Obecnie: 1
ID: 2 | Jeans (32) | Obecnie: 15
-----
Podaj ID produktu do uzupełnienia (lub 0 dla nowego): 0
Nazwa: Kurtka
Rozmiar: XL
Płeć (M/F): M
Cena: 500
Ilość: 10
Nowy towar przyjęty na stan.
```

Rysunek 6.5: Proces dodawania nowego produktu (Kurtka)

```
C:\Users\Wikt\or\Desktop\pro X + v
=== PRZYJĘCIE TOWARU ===
ID: 1 | T-Shirt (L) | Obecnie: 1
ID: 2 | Jeans (32) | Obecnie: 15
ID: 3 | Kurtka (XL) | Obecnie: 10
-----
Podaj ID produktu do uzupełnienia (lub 0 dla nowego):
```

Rysunek 6.6: Stan magazynu po aktualizacji - widoczna nowa pozycja

Przedstawiony scenariusz ilustruje pełny proces aktualizacji magazynu, począwszy od przeglądu stanu początkowego, aż do potwierdzenia wprowadzenia nowego produktu do systemu.

6.1.2 Wyświetlenie historii zamówień

Widok historii zamówień umożliwia sprzedawcy wgląd w zrealizowane transakcje poszczególnych klientów. Informacje te pozwalają na kontrolę sprzedaży oraz weryfikację poprawności działania systemu zamówień.

```
C:\Users\Wikt\Deskto\pro X + v
===== HISTORIA ZAMÓWIEŃ =====
ZAM. #1 | Wiktor Nicpoń (Wiktor@shop.pl)
KWOTA: 150,00 zł | STATUS: Złozone
-----
ZAM. #2 | Jan Kowalski (Jan@shop.pl)
KWOTA: 270,00 zł | STATUS: Złozone
-----
```

Rysunek 6.7: Historia zamówień poszczególnych klientów

6.1.3 Zarządzanie klientami

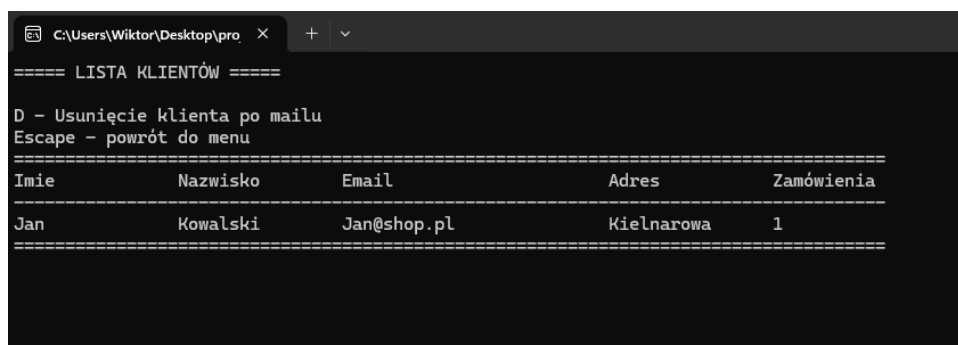
Administrator posiada uprawnienia do usuwania użytkowników. Poniżej zaprezentowano listę klientów przed operacją (Rysunek 6.8), proces usuwania klienta (Rysunek 6.9) oraz zaktualizowaną listę (Rysunek 6.10).

```
C:\Users\Wikt\Deskto\pro X + v
===== LISTA KLIENTÓW =====
D - Usunięcie klienta po mailu
Escape - powrót do menu
=====
Imie          Nazwisko      Email          Adres          Zamówienia
-----
Wiktor        Nicpoń       Wiktor@shop.pl Rzeszów        1
Jan           Kowalski     Jan@shop.pl    Kielnarowa     1
=====
```

Rysunek 6.8: Lista klientów przed usunięciem

```
C:\Users\Wikt\Deskto\pro X + v
===== LISTA KLIENTÓW =====
D - Usunięcie klienta po mailu
Escape - powrót do menu
=====
Imie          Nazwisko      Email          Adres          Zamówienia
-----
Wiktor        Nicpoń       Wiktor@shop.pl Rzeszów        1
Jan           Kowalski     Jan@shop.pl    Kielnarowa     1
=====
Podaj email klienta do usunięcia: Wiktor@shop.pl
Klient usunięty.
```

Rysunek 6.9: Usuwanie użytkownika z bazy po adresie email



```
==== LISTA KLIENTÓW ====
D - Usunięcie klienta po mailu
Escape - powrót do menu
=====
Imie      Nazwisko   Email      Adres      Zamówienia
-----
Jan       Kowalski  Jan@shop.pl Kielnarowa  1
=====
```

Rysunek 6.10: Zaktualizowana lista klientów (brak użytkownika Wiktor)

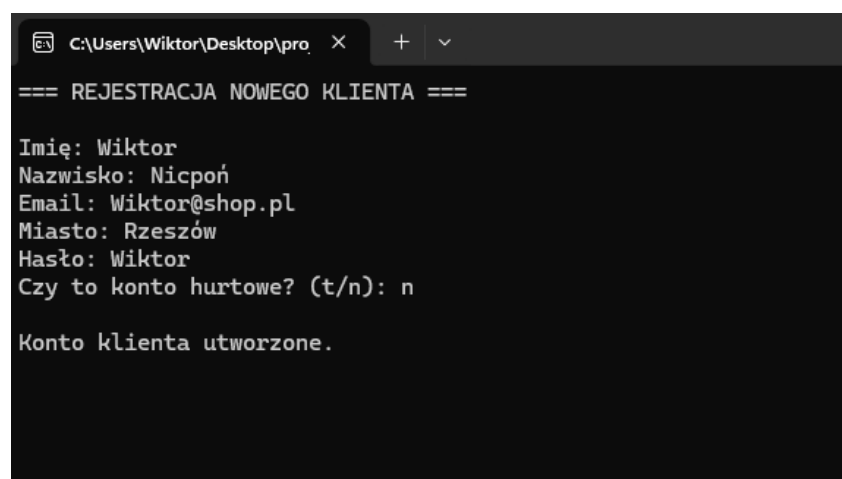
Proces usuwania klienta z systemu został zaprojektowany w sposób prosty i jednoznaczny. Po wykonaniu operacji lista użytkowników jest natychmiast aktualizowana, co potwierdza poprawne działanie mechanizmu administracyjnego.

6.2 Klient - Scenariusze zakupowe

Aplikacja obsługuje zarówno klientów detalicznych, jak i hurtowych. W tej części zaprezentowano typowe scenariusze zakupowe realizowane przez użytkowników systemu. Przebieg procesu różni się w zależności od rodzaju konta, co pozwala na zastosowanie odmiennych zasad cenowych.

6.2.1 Scenariusz Klienta Detalicznego

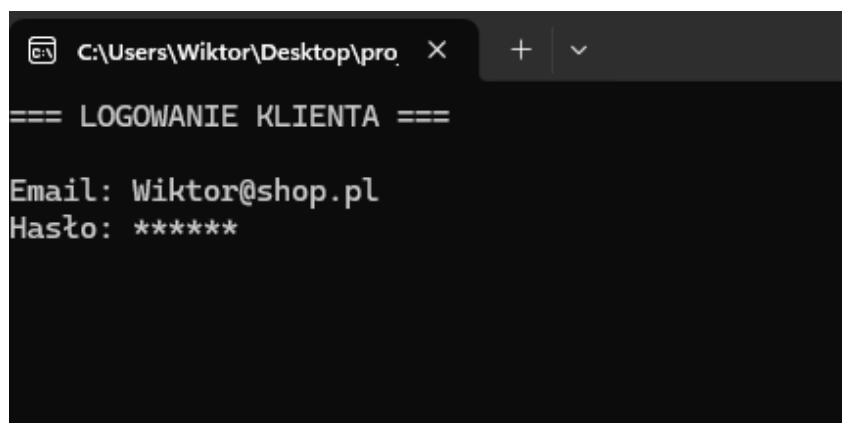
Nowy użytkownik rozpoczyna od rejestracji (Rysunek 6.11). Wybór opcji "n" przy pytaniu o konto hurtowe tworzy standardowe konto detaliczne.



```
=== REJESTRACJA NOWEGO KLIENTA ===
Imię: Wiktor
Nazwisko: Nicpoń
Email: Wiktor@shop.pl
Miasto: Rzeszów
Hasło: Wiktor
Czy to konto hurtowe? (t/n): n
Konto klienta utworzone.
```

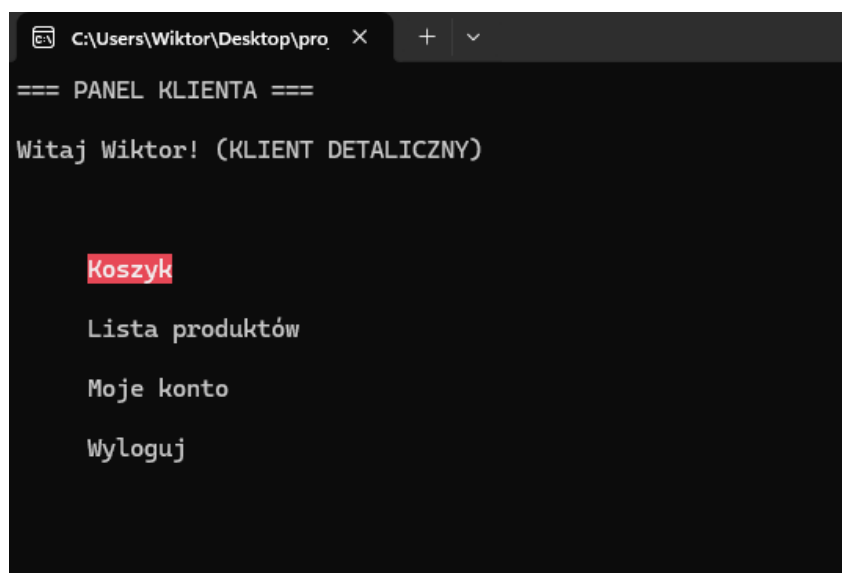
Rysunek 6.11: Rejestracja klienta detalicznego

Następnie następuje logowanie (Rysunek 6.12).



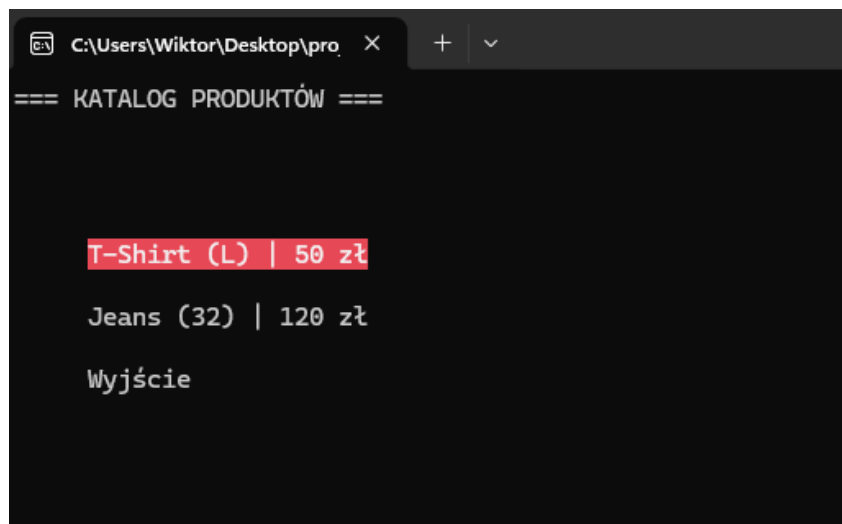
Rysunek 6.12: Logowanie do systemu

Po zalogowaniu klient widzi swój panel (Rysunek 6.13). Brak oznaczeń o zniżkach potwierdza status detaliczny.

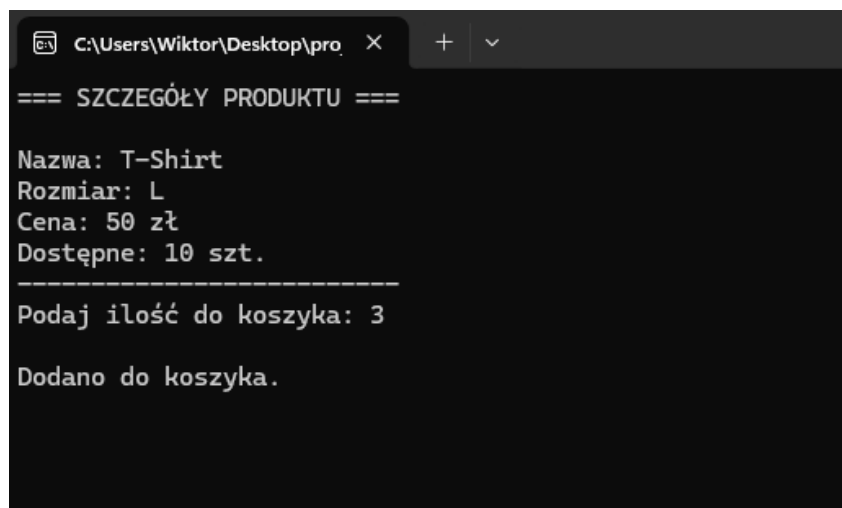


Rysunek 6.13: Panel klienta detalicznego

Klient przegląda katalog (Rysunek 6.14), wybiera produkt i dodaje go do koszyka (Rysunek 6.15). Ceny są standardowe.

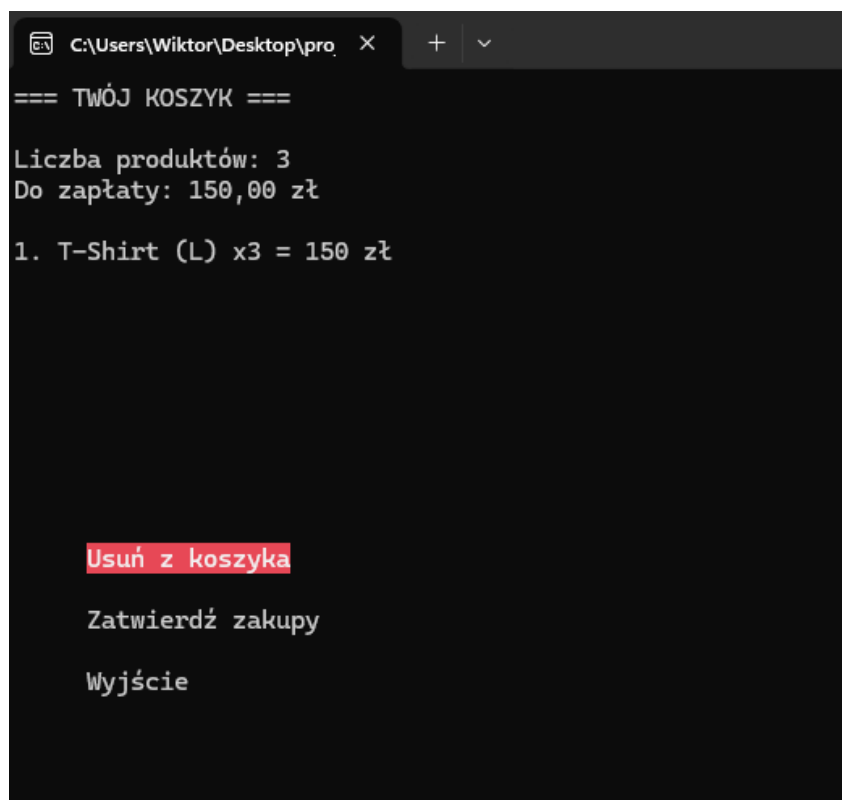


Rysunek 6.14: Katalog produktów

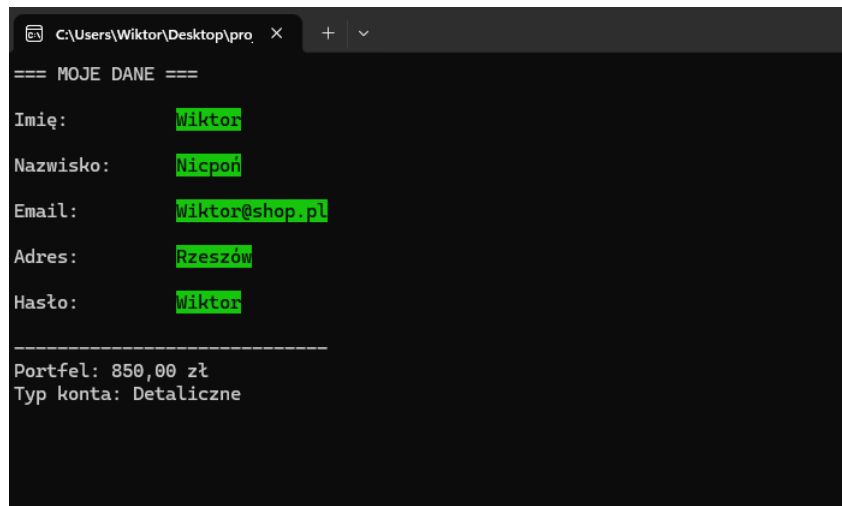


Rysunek 6.15: Dodawanie produktu do koszyka

W koszyku widoczne jest podsumowanie bez naliczonego rabatu (Rysunek 6.16).



Rysunek 6.16: Koszyk klienta detalicznego



Rysunek 6.17: Dane klienta detalicznego (dane wyświetlone po zakupie w sklepie)

Przedstawiony scenariusz pokazuje standardowy proces zakupowy bez naliczania dodatkowych rabatów. Wszystkie ceny prezentowane są w wartości podstawowej, co jednoznacznie identyfikuje użytkownika jako klienta detalicznego.

6.2.2 Scenariusz Klienta Hurtowego

System rozróżnia klientów biznesowych. Rejestracja konta hurtowego wymaga potwierdzenia opcji (Rysunek 6.18).

```
C:\Users\Wikt\or\Desktop\pro  X  +  v

=== REJESTRACJA NOWEGO KLIENTA ===

Imię: Jan
Nazwisko: Kowalski
Email: Jan@shop.pl
Miasto: Kielnarowa
Hasło: Jan
Czy to konto hurtowe? (t/n): t

Konto klienta utworzone.
```

Rysunek 6.18: Rejestracja konta hurtowego

Panel hurtownika (Rysunek 6.19) wyraźnie informuje o aktywnejniżce -10%.

```
C:\Users\Wikt\or\Desktop\pro  X  +  v

=== PANEL KLIENTA ===

Witaj Jan! (KLIENT HURTOWY -10%)

Koszyk

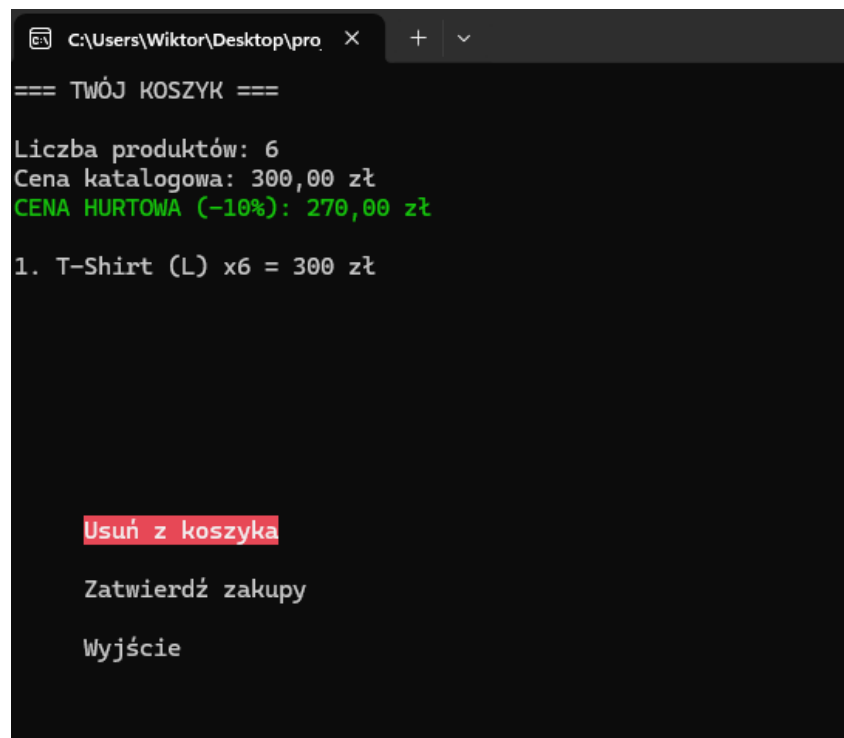
Lista produktów

Moje konto

Wyloguj
```

Rysunek 6.19: Panel klienta hurtowego ze zniżką

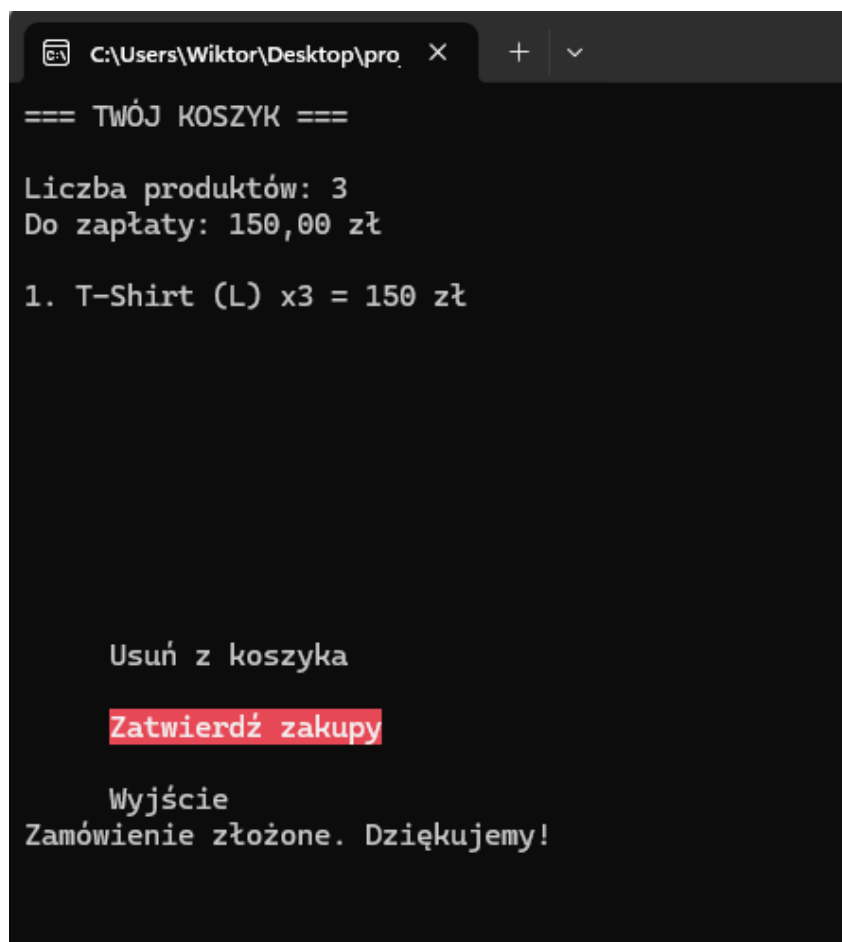
Podczas zakupów system automatycznie przelicza ceny. W koszyku hurtownika (Rysunek 6.20) cena końcowa jest obniżona i wyróżniona kolorem zielonym.



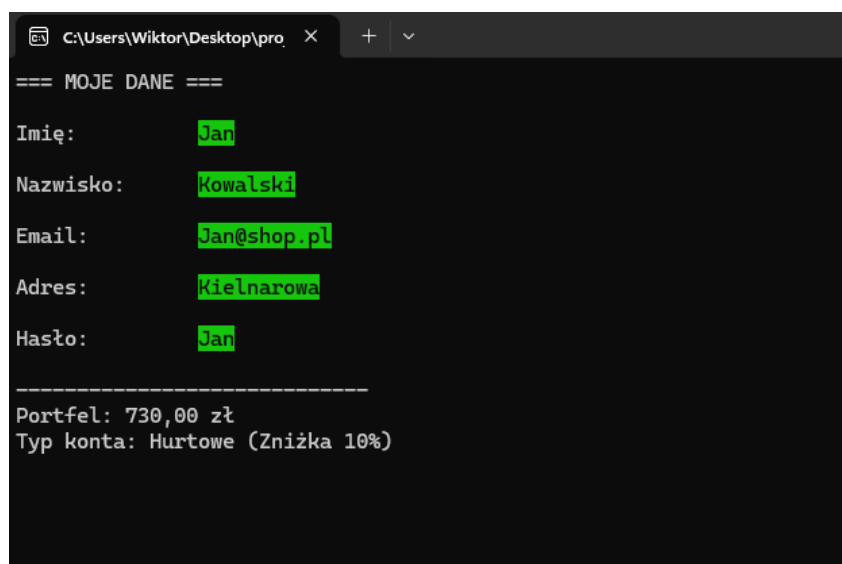
```
=== TWÓJ KOSZYK ===  
  
Liczba produktów: 6  
Cena katalogowa: 300,00 zł  
CENA HURTOWA (-10%): 270,00 zł  
  
1. T-Shirt (L) x6 = 300 zł  
  
Usuń z koszyka  
Zatwierdź zakupy  
Wyjście
```

Rysunek 6.20: Koszyk z naliczonym rabatem hurtowym

Zatwierdzenie koszyka kończy transakcję (Rysunek 6.21).



Rysunek 6.21: Potwierdzenie złożenia zamówienia



Rysunek 6.22: Dane klienta hurtowego (dane wyświetlone po zakupie w sklepie)

W przypadku klienta hurtowego system automatycznie uwzględnia rabat procentowy podczas obliczania wartości zamówienia. Informacja o obniżonej cenie jest wyraźnie widoczna w interfejsie, co zwiększa przejrzystość procesu zakupowego.

Rozdział 7

Podsumowanie

7.1 Wnioski z realizacji projektu

Projekt „System Zarządzania Sklepem” został pomyślnie zrealizowany zgodnie z założonymi wymaganiami funkcjonalnymi i нефункциональными. Opracowana aplikacja konsolowa umożliwia kompleksową obsługę podstawowych procesów związanych z funkcjonowaniem sklepu odzieżowego. System w poprawny sposób realizuje kluczowe operacje, takie jak zarządzanie asortymentem, obsługa zamówień oraz administracja użytkownikami.

Aplikacja spełnia wszystkie założenia funkcjonalne, poprawnie obsługując:

- zarządzanie bazą towarową (operacje typu CRUD) przez sprzedawcę,
- administrację kontami użytkowników, w tym usuwanie klientów z systemu,
- procesy sprzedażowe z uwzględnieniem różnych ról użytkowników (klient detaliczny oraz hurtowy).

Przeprowadzone testy potwierdziły poprawność działania systemu oraz prawidłową aktualizację danych zapisywanych w plikach tekstowych. Zastosowane rozwiązania zapewniają stabilność działania aplikacji oraz spójność przechowywanych informacji.

7.2 Plany rozbudowy

W przyszłości planowana jest dalsza rozbudowa aplikacji w celu zwiększenia jej funkcjonalności oraz komfortu użytkowania. Jednym z kierunków rozwoju jest wdrożenie graficznego interfejsu użytkownika w technologii WPF, co pozwoli na bardziej intuicyjną obsługę systemu. Dodatkowo rozważana jest migracja mechanizmu przechowywania danych z plików tekstowych do relacyjnej bazy danych SQL, co umożliwi lepszą skalowalność oraz integrację z innymi systemami.

Bibliografia

1. Dokumentacja Microsoft .NET, <https://learn.microsoft.com/pl-pl/dotnet/>.
2. Matulewski J., C: lekcje programowania: praktyczna nauka programowania dla platform .NET i .NET Core, Helion, Gliwice 2021
3. Kurs edukacyjny wideo dotyczący podstaw języka C#, dostęp online: https://youtu.be/zP6BAsU1D_A.